



USO DE LA PLATAFORMA DE CÓMPUTO MATLAB

1. Introducción a MATLAB

MATLAB (MATrix LABoratory) es un entorno de programación y un software de cálculo numérico ampliamente utilizado en ingeniería, matemáticas y ciencias aplicadas. Su fortaleza radica en la facilidad para manipular matrices, resolver ecuaciones matemáticas, procesar señales e imágenes, desarrollar modelos de simulación y realizar análisis de datos.

A diferencia de otros lenguajes de programación como Python o C, MATLAB está optimizado para operaciones matriciales, lo que lo hace especialmente eficiente en cálculos científicos y técnicos. Además, cuenta con una amplia variedad de herramientas y bibliotecas para diferentes aplicaciones, desde el aprendizaje automático hasta el control de sistemas dinámicos.

¿Por qué usar MATLAB?

Algunas de las razones por las que MATLAB es popular en el ámbito académico y profesional incluyen:

1. **Sintaxis simple y clara**, lo que facilita el aprendizaje y la implementación rápida de algoritmos.
2. **Gráficos y visualización de datos** con herramientas integradas para representar información de forma intuitiva.
3. **Amplia gama de funciones matemáticas y científicas**, optimizadas para alto rendimiento.
4. **Compatibilidad con otros lenguajes** como Python, C, C++ y Java.
5. **Herramientas específicas para diversas disciplinas**, como procesamiento de señales, optimización, aprendizaje automático y control de sistemas.

2. Instalación y Acceso a MATLAB

Antes de comenzar a programar en MATLAB, es importante familiarizarse con su entorno y aprender algunas operaciones básicas.

Instalación y Acceso a MATLAB

Para usar MATLAB, puedes:

1. **Usar MATLAB Online (recomendado)**: Si dispones de una licencia, puedes acceder a la versión en la nube sin necesidad de instalación en <https://matlab.mathworks.com>. En caso no dispongas de una licencia, podrás hacer uso de Matlab por un total de 20 horas (Matlab *On-line free*) cada mes. Para emplear Matlab *for free*, inicia sesión y luego da click en “**Use**

now for free". En caso no tengas una cuenta, deberás crearla antes de poder emplear Matlab.

2. **Instalar MATLAB en tu PC:** Debes descargarlo desde la página oficial de MathWorks (<https://www.mathworks.com>), dando click en "**Obtenga Matlab**" y seguir el proceso indicado. En caso de que no dispongas de licencia, podrás ejecutar Matlab por un periodo de 30 días.
3. **Emplear la versión open-source Octave:** Debes descargarla desde la página oficial de GNU Octave (<https://octave.org/>) y seguir el proceso de instalación. Puedes descargar la guía de instalación de Octave disponible en la plataforma Ude@. Útil si no tienes una licencia de MATLAB y la versión On-line free no es suficiente.

Explorando la Interfaz de MATLAB

Al abrir MATLAB, encontrarás varias secciones clave en la pantalla:

- **Command Window:** Donde puedes escribir comandos y ejecutar código.
- **Workspace:** Muestra las variables que has definido en la sesión actual.
- **Current Folder:** Indica la carpeta en la que estás trabajando y donde se guardan los archivos.
- **Editor:** Permite escribir y guardar scripts y funciones en archivos .m.

La Figura 1 muestra los principales elementos del entorno integrado de desarrollo IDE de MATLAB:

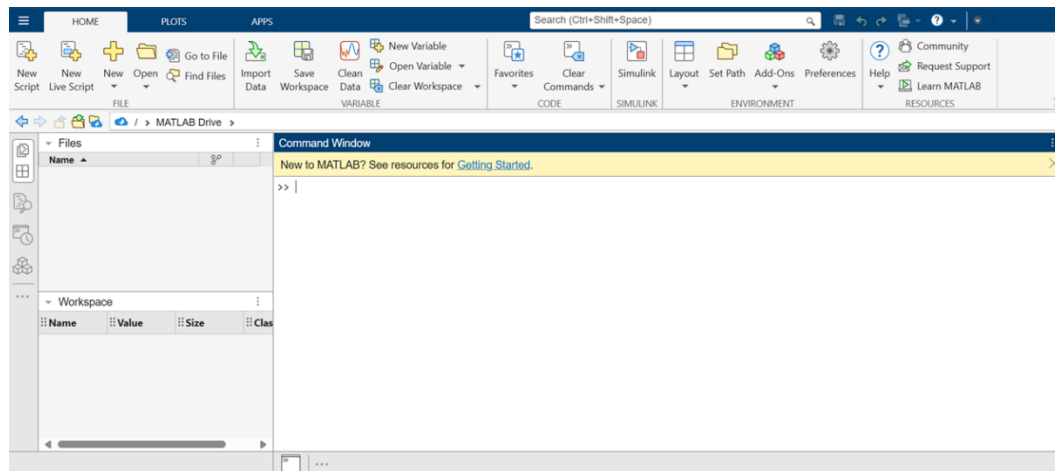


Figura 1. Entorno integrado de desarrollo en Matlab (Versión online).

A continuación, se describen cada uno de los elementos principales:

- **Command window (Command prompt):** Lugar del IDE donde se ejecutan todas las instrucciones y programas. Para introducir un comando, basta con escribirlo en la ventana de comandos y luego presionar *enter* como se ilustra en la Figura 2.
- **Command history:** Muestra los últimos comandos ejecutados en el **command prompt**. Los comandos desplegados allí se pueden ejecutar dando doble *click* sobre estos. La Figura 3 muestra la ventana de historial.

- **Current directory:** Directorio de trabajo actual.

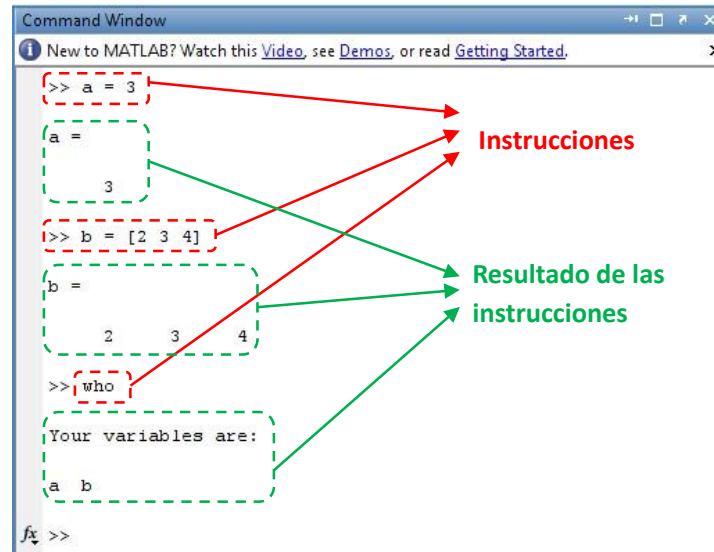


Figura 2. Command Window en MATLAB

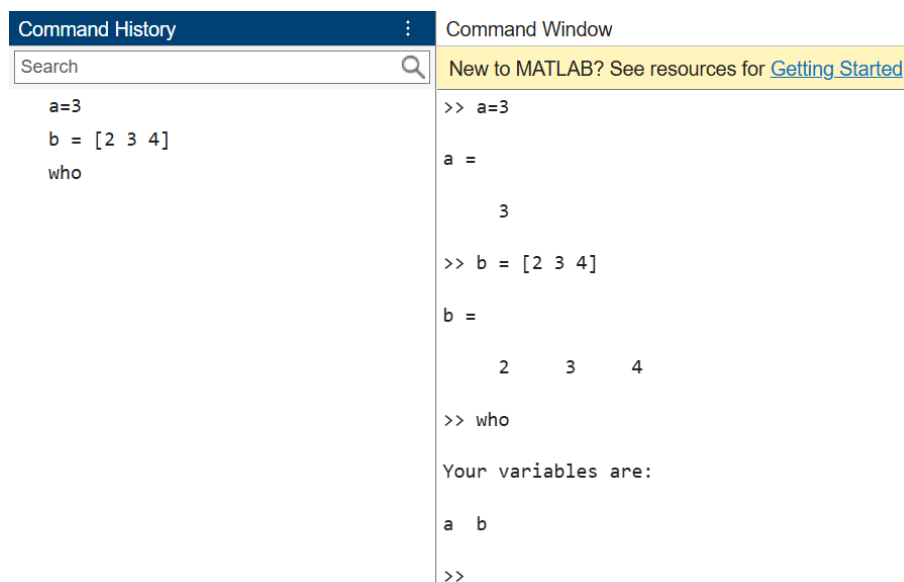


Figura 3. Historial de comandos ejecutados.

Finalmente, un comando sumamente útil es **help**, cuya sintaxis se muestra a continuación:

>> **help** comando

Este comando muestra en el prompt como es el funcionamiento de un comando específico. Cuando el comando **help** se escribe sin ningún comando, muestra una lista de tópicos generales en los cuales se agrupan los diferentes comandos de Matlab.

3. Operaciones numéricas en MATLAB

En MATLAB, a diferencia de los lenguajes de programación comunes, no es necesaria la declaración de variables. Así mismo, con Matlab es posible manejar todo tipo de variables ya sea enteras, reales o caracteres, entre otras. A continuación, se declaran algunas variables de los diferentes tipos en Matlab (note que no hay que especificar el tipo de dato, simplemente Matlab lo infiere):

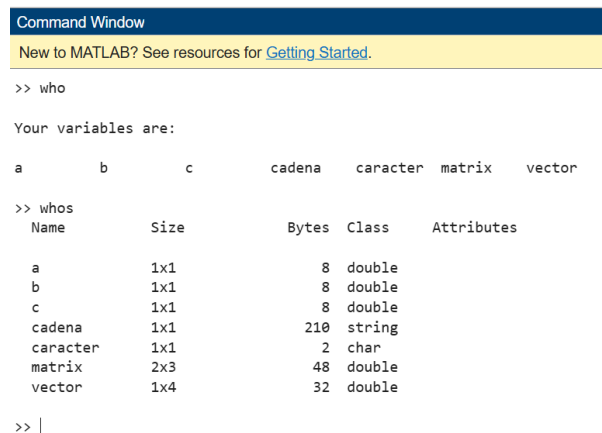
Experimento. Ejecute las siguientes líneas en MATLAB y observe qué sucede:

```
>> a = 2
>> b = 2.3, c = -3;
>> vector = [1,2,3,4]
>> matrix = [2,3,4; 1,0,-1]
>> caracter = 'C'
>> cadena = "Curso de Circuitos Electricos"
```

Tenga en cuenta que después de especificar una sentencia, ésta será mostrada junto con su resultado. Si quiere omitir su despliegue, coloque un punto y coma (;) después de la sentencia.

Experimento. Ejecute algunas de las líneas anteriores, poniendo un punto y coma al final.

Para ver que variables se tienen en el *workspace* se hace uso del comando **who** o **whos**.



```
Command Window
New to MATLAB? See resources for Getting Started.

>> who

Your variables are:

a          b          c          cadena    caracter  matrix    vector

>> whos

Name      Size      Bytes    Class    Attributes

a         1x1         8    double
b         1x1         8    double
c         1x1         8    double
cadena    1x1       210    string
caracter  1x1         2     char
matrix    2x3        48    double
vector    1x4        32    double

>> |
```

Figura 4. Variables en el Workspace

La diferencia entre uno y otro comando radica en el hecho en que en el primer caso solo se muestran las variables existentes en el *workspace*, mientras que, en el segundo caso, además de las variables, se muestran el tipo al cual pertenecen.

Para eliminar una variable del *workspace* se emplea el comando **clear** tal y como se muestra a continuación:

```
>> clear variable1 variable2 ... variableN
```



Donde los argumentos (**variable1**, **variable2**, ... , **variableN**) son las variables para eliminar. Si se emplea el comando sin argumentos se eliminan todas las variables previamente declaradas.

Experimento. Elimine algunas de las variables previamente creadas en MATLAB y verifique que hayan sido efectivamente eliminadas ¿Cómo lo haría usted?.

Las operaciones matemáticas elementales se muestran en la Tabla 1.

Signo	Operación
+	Suma
-	Resta
*	Multiplicación
/	División
^	Potenciación

Tabla 1. Operaciones en MATLAB

Cuando se declara una variable numérica esta puede tener diferentes formatos. La Tabla 2 muestra los formatos disponibles.

Formato	Descripción
format short	Formato con 4 dígitos después del punto decimal. Por ejemplo 3.1416.
format long	Formato con 14 o 15 dígitos después del punto decimal cuando la variable es <i>double</i> o 7 dígitos cuando el dato es <i>single</i> . Por ejemplo: 3.141592653589793.
format short e	Notación exponencial con 4 dígitos después del punto decimal. Por ejemplo: 3.1416e+000.
format long e	Formato con 14 o 15 dígitos después del punto decimal cuando se tiene precisión <i>double</i> o 7 dígitos cuando la precisión es <i>single</i> . Por ejemplo: 3.141592653589793e+000.
Format rat	Aproximación racional. Por ejemplo: 355/113

Tabla 2. Formatos que soportan las variables

Experimento. Emplee distintos formatos para visualizar el valor de pi:

```
>> format long
>> pi
>> format rat
>> pi
>> format short
```

La Tabla 3 muestra algunas funciones numéricas comúnmente empleadas en MATLAB.

Grupo	Funciones
Exponenciales y logarítmicas	exp(x), log(x), log2(x), log10(x), sqrt(x)
Funciones trigonométricas	sin(x), cos(x), tan(x), asin(x), acos(x), atan(x), atan2(x)
Funciones hiperbólicas	sinh(x), cosh(x), tanh(x), asinh(x), acosh(x), atanh(x)
Otras funciones	abs(x), int(x), round(x), sign(x)

Tabla 3. Funciones básicas

Además de los números reales con Matlab es posible manipular cantidades complejas.

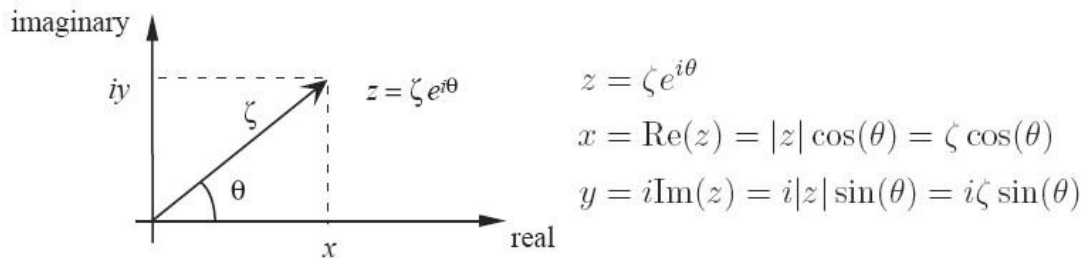


Imagen 9 Representación de un número complejo en el plano

Para manipular valores complejos, MATLAB dispone de la letra **i** o la **j**. Sin embargo, lo anterior no impide que **i** o **j** puedan ser usadas para otros propósitos, pues es posible su reasignación.

Además de las operaciones normales sobre cantidades numéricas (+, -, *, /, ^), MATLAB brinda otra serie de operaciones especialmente útiles para números complejos. La Tabla 4 define operaciones con números complejos.

Función	Operación
abs	Calcula la magnitud de un número. Así, abs(z) es equivalente a sqrt(real(z)^2 + imag(z)^2).
angle	Angulo de un número complejo en notación de Euler.
exp	Retorna el exponencial de un valor
conj	Complejo conjugado del número
imag	Extrae la parte imaginaria de un número complejo.
real	Extrae la parte real de un número complejo.

Tabla 4. Funciones para números complejos

Experimento. Realice operaciones con números complejos:

```
>> a = 5 + pi*i
>> real(a)
>> imag(a)
```



4. Matrices y vectores en MATLAB

MATLAB permite el trabajo y la manipulación de matrices. La Tabla 5 muestra cómo crear matrices y vectores asignando manualmente valores.

Comando Matlab	Observaciones
<code>v= [7 3 9]</code>	Declaración de un vector fila.
<code>v= [7,3,9]</code>	Declaración de un vector fila (Forma alternativa).
<code>w= [2;-6;1]</code>	Declaración de un vector columna.
<code>M= [1 2 3;5 7 11;17 17 9]</code>	Declaración de una matriz. Los elementos de las filas son separados por espacios o comas (,), mientras que los elementos de las columnas son separados por punto y coma (;).
<code>MV= []</code>	Crea una matriz vacía

Tabla 5. Vectores y Matrices

Experimento. Ejecute los comandos de la Tabla 5 e interprete los resultados.

Es posible crear vectores y matrices por medio de funciones incorporadas. Por ejemplo, la función **linspace** crea un vector cuyos elementos están uniformemente espaciados.

Sintaxis:

```
x = linspace(startValue,endValue)
x = linspace(startValue,endValue,nelements)
```

El número de elementos creados se encuentra entre *startValue* y *endValue* siendo 100 el valor por defecto para *nelements* cuando éste no se especifica.

Experimento. Ejecute los siguientes comandos e interprete los resultados:

```
>> v1 = linspace(1,10)
>> v2 = linspace(1,10,10)
>> v3 = linspace(1,10,5)
```

Otras funciones para crear vectores y matrices con valores iniciales se muestran en Tabla 6.

Función	Significado
ones(nrows,ncols)	Crea una matriz de unos de <i>nrows x ncols</i> . Si <i>ncols</i> no es especificado la matriz creada será cuadrada (<i>nrows x nrows</i>).
zeros(nrows,ncols)	Crea una matriz de ceros de <i>nrows x ncols</i> . Si <i>ncols</i> no es especificado la matriz creada será cuadrada (<i>nrows x nrows</i>).



eye(n) eye(nrows,ncols)	En el primer caso crea una matriz identidad de $n \times n$. Cuando se especifican las filas y las columnas de la matriz, crea una matriz no cuadrada con 1's en su diagonal principal.
rand(nrows,ncols)	Generación de una matriz de elementos $nrows \times ncols$ aleatorios.

Tabla 6. Crear matrices con funciones predefinidas

Experimento. Ejecute los siguientes comandos e interprete los resultados:

```
>> C = eye(5)
>> D = eye(3,5)
>> s = ones(1,4)
>> t = zeros(3,1)
>> D = ones(3,3)
>> E = ones(2,4)
>> F = rand(1,3)
```

a. Accediendo a matrices y vectores

Para acceder a los diferentes elementos de vectores y matrices previamente creados, se emplea la notación subíndice. La Tabla 7 muestra cada caso:

Notación	Significado
v(i)	Permite acceder al i-ésimo elemento del vector v. El valor de i está entre 1 y el número de elementos del vector, es decir, si v tuviera 10 elementos, el valor de i estaría entre 1 y 10, siendo por ejemplo, v(5) el quinto elemento.
m(i,j)	Selecciona el elemento que se encuentra en la fila i y en la columna j. El valor de i deberá estar entre 1 y el número de filas, el valor de j deberá estar entre 1 y el número de columnas. Cualquier violación a estos rangos generará un mensaje de error.

Tabla 7. Acceso a los elementos de una matrix o vector

Experimento. Ejecute los siguientes comandos e interprete los resultados:

```
>> A = [1 2 3; 4 5 6; 7 8 9]
>> b = A(3,2)
>> c = A(1,1) + 2 - A(2,3)
>> A(2,3) = -10
>> A(4,4) = 5
>> v = [1 2 3 4]
>> v(2) * v(3)
>> 70 * v(5)
```




b. Notación dos puntos (Colon notation)

La **notación dos puntos (:)** es una notación muy importante y efectiva en el uso de MATLAB. Los dos puntos (:) también suelen ser usados como operador o como comodín. Dentro de los principales usos de los dos puntos tenemos:

- La creación de vectores.
- Para referirse o extraer rangos de elementos de matrices.

La sintaxis básica de esta notación se muestra a continuación:

- Opción 1: startValue:endValue
- Opción 2: startValue:increment:endValue

Donde:

- startValue: Valor inicial.
- endValue: Valor final.
- increment: Incremento.

Experimento. Ejecute los siguientes comandos e interprete los resultados:

```
>> s = 1:4
>> t = 0:0.1:0.5
>> s = (1:4)'
>> t = (0:0.1:0.5)'
>> A = [1 2 3; 4 5 6; 7 8 9]
>> A(:,1)
>> A(:,3)
>> A(2,:)
>> A(2:3,1)
>> A(1:2,2:3)
>> A(2:3,1:2)
>> A = zeros(6,6), A(2:5,2:5) = ones(4,4), A(3:4,3:4) = zeros(2,2)
>> A = rand(2,4), B = A(:)
```

c. Operaciones con matrices

Sean **u** y **v** vectores (o matrices) y **k** un escalar, MATLAB define una serie de operaciones para matrices las cuales se resumen en las Tablas 8, 9 y 10:

Operaciones de vectores y matrices con escalares	
Operación	Observaciones
v + k	Suma
v - k	Resta



$v * k$	Multiplicación
v / k	División de cada elemento de v por k
$k ./ v$	Divide k por cada elemento de v
$v.^k$	Eleva cada componente de v a la k
$k.^v$	K elevado a cada componente de v.

Tabla 8. Operaciones básicas con matrices

Operaciones con vectores y matrices	
Operación	Observaciones
+	Suma
-	Resta
*	Multiplicación matricial
.*	Producto elemento a elemento
^	Potenciación
.^	Elevar a una potencia elemento a elemento
/	División-izquierda
\	División-derecha
./ y .\	División elemento a elemento
Matriz transpuesta	$B=A'$, en complejos calcula la transpuesta conjugada. $B=A'$ calcula la transpuesta

Tabla 9. Operadores para uso de vectores y matrices

Función	Descripción
sum(v)	Suma los elementos de un vector.
prod(v)	Producto de los elementos de un vector.
dot(v,w)	Producto escalar de vectores.
cross(v,w)	Producto vectorial de vectores.
mean(v)	Media de un vector.
diff(v)	Vector cuyos elementos son la resta de los elementos de v.
[y,k]=max(v)	Valor máximo de las componentes de un vector (k indica la posición).
min(v)	Valor mínimo.
min(min(M))	Valor máximo de una matriz M.
[n,m]=size(M)	Numero de filas y columnas de la matriz M.
B=inv(M)	Matriz inversa de M.
rank(M)	Rango de M.
norm(M)	Norma de una matriz (máximo de los valores absolutos de los elementos de M).



flipud(M)	Reordena la matriz, haciéndola simétrica respecto de un eje horizontal.
fliplr(M)	Reordena la matriz, haciéndola simétrica respecto de un eje vertical.

Tabla 10. Funciones predefinidas para operaciones con vectores y matrices

5. Gráficas 2D en Matlab

Con Matlab es posible realizar gráficas bidimensionales empleando la función **plot** cuya sintaxis se muestra a continuación:

- `plot(x,y)`
- `plot(xdata,ydata,symbol)`
- `plot(x1,y1,x2,y2,...)`
- `plot(x1,y1,symbol1,x2,y2,symbol2,...)`

Donde:

- **x, y:** vectores de la abscisa y la ordenada. Deben tener el mismo tamaño.
- **xdata, ydata:** vectores de la abscisa y la ordenada.
- **symbol:** símbolo que tendrá la línea dibujada.
- **x1, y1, x2, y2, ...:** pares abscisa-ordenada para el caso en el cual se van a hacer varios gráficos en el mismo eje.
- **Symbol1, symbol2, ...:** símbolos que tendrá cada una de las líneas dibujadas, en caso de tener varias líneas en el mismo eje.

Para dibujar las curvas se emplean las combinaciones de color, símbolos y tipo de líneas mostradas en la Tabla 11.

Color		Símbolo		Tipo de línea	
y	amarillo	.	Punto.	-	solido.
m	magenta	o	Circulo.	:	Punteado.
c	cyan	x	Marca x.	-.	Dashdot.
r	rojo	+	Marca +.	--	Dashed.
g	verde	*	Marca estrella.		
b	azul	s	Cuadrado.		
w	blanco	d	Diamante.		
k	negro	v	Triangulo abajo.		
		^	Triangulo arriba.		
		<	Triangulo izquierda.		
		>	Triangulo derecha.		
		p	Pentagrama.		
		h	Hexagrama.		

Tabla 11. Caracteres comunes para personalizar gráficas

Experimento. Reproduzca los ejemplos de la Tabla 12:

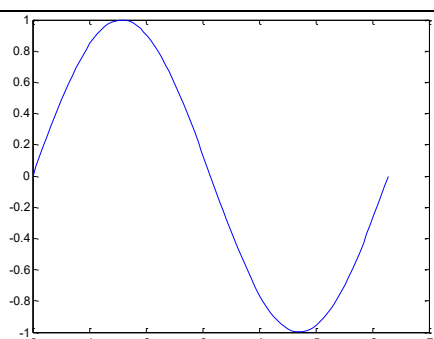
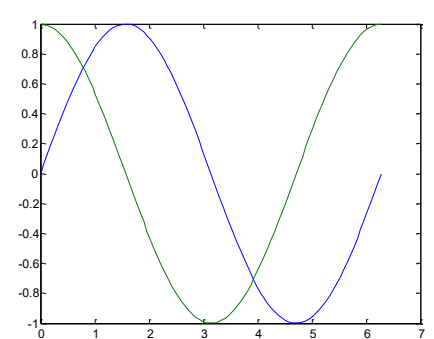
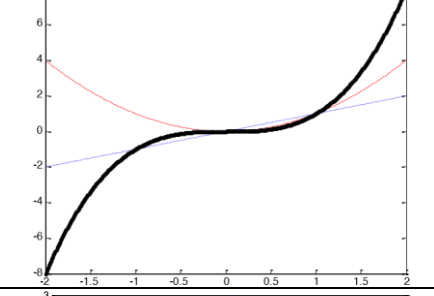
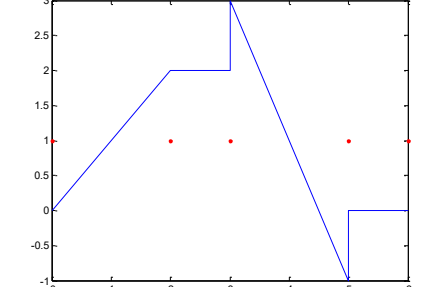
Comandos	Resultado
<pre>>> x = linspace(0, 2*pi); >> y = sin(x); >> plot(x, y);</pre>	
<pre>>> x = linspace(0, 2*pi); >> y1 = sin(x); y2 = cos(x); >> plot(x, y1, x, y2);</pre>	
<pre>>> x = -2:0.01:2; >> y1 = x; >> y2 = x.^2; >> y3 = x.^3; >> plot(x,y1,':',x,y2,'r--',x,y3,'k.-');</pre>	
<pre>>> x = [0 2 2 3 5 5 6]; >> y = [0 2 3 3 5 5 6]; >> z = [1 1 1 1 1 1 1]; >> plot(x,y,x,z,'r.');</pre>	

Tabla 12. Ejemplos de gráficas usando comando plot

Es posible manejar diferentes escalas cuando se está realizando una u otra gráfica particular. Esto se realiza por medio de funciones que tienen la misma sintaxis del **plot** como se muestra en la Tabla 13.

Función	Escalamiento de los ejes
loglog	$\log_{10}(y)$ versus $\log_{10}(x)$
plot	Lineal y versus x
semilogx	Lineal y versus $\log_{10}(x)$
semilogy	$\log_{10}(y)$ versus x

Tabla 13. Comandos para el escalamiento de ejes

Experimento. Ejecute los siguientes comandos e interprete los resultados:

```
>> x = linspace(0,3);
>> y = 10*exp(-2*x);
>> plot(x,y);
```

```
>> x = linspace(0,3);
>> y = 10*exp(-2*x);
>> semilog(x,y);
```

Colocando varias gráficas en una misma figura

La función **subplot** es usada para crear una matriz de gráficos en una misma figura. Su sintaxis se muestra a continuación: `subplot(nrows,ncols,thisPlot)`. La matriz de gráficos será de **nrows** por **ncols**. Para seleccionar la posición en la matriz se emplea el último argumento (**thisPlot**). La Figura 5 muestra una matriz de gráficos de 2x2 con los diferentes valores para **nrows**, **ncols** y **thisPlot**.

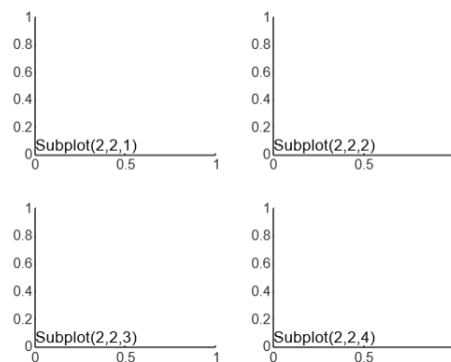


Figura 5. Ejemplo de subplot

Experimento. Ejecute los siguientes comandos e interprete los resultados:

```
>> subplot(2,2,[1 3])
>> subplot(2,2,2)
>> subplot(2,2,4)
```

```
>> subplot(2,2,1:2)
>> subplot(2,2,3)
>> subplot(2,2,4)
```



Después de que se usa el comando **subplot** para la selección del gráfico, se procede a usar el comando **plot** para graficar los datos en el eje seleccionado.

Experimento. Ejecute los siguientes comandos e interprete los resultados:

```
>> t = linspace(0,1,1000);  
>> subplot(2,2,1);  
>> f = 1; plot(t,sin(2*pi*f*t)); title(sprintf('sin(2*pi*%d*t)',f));  
>> subplot(2,2,2);  
>> f = 2; plot(t,sin(2*pi*f*t)); title(sprintf('sin(2*pi*%d*t)',f));  
>> subplot(2,2,3);  
>> f = 3; plot(t,sin(2*pi*f*t)); title(sprintf('sin(2*pi*%d*t)',f));  
>> subplot(2,2,4);  
>> f = 4; plot(t,sin(2*pi*f*t)); title(sprintf('sin(2*pi*%d*t)',f));
```

Para hacer las gráficas más amigables, MATLAB suele proporcionar una serie de funciones las cuales se resumen en la Tabla 14.

Función	Operación realizada
axis	Definir límites y estilos en los ejes.
grid	Dibuja líneas de cuadrícula.
gtext	Añade texto en la localización determinada por un click del mouse.
legend	Crea una legenda para identificar símbolos y tipos de línea cuando múltiples curvas son dibujadas.
text	Coloca texto en una localización (x,y).
xlabel	Establece título del eje x.
ylabel	Establece título del eje y.
title	Añade título a la gráfica.

Tabla 14. Funciones útiles para darle formato a las gráficas

En el curso de Circuitos Eléctricos es común emplear señales singulares tales como el escalón unitario, el impulso y la rampa.

Experimento. Ejecute los siguientes comandos e interprete los resultados:

```
>> t = -10:0.01:10;  
>> impulso_unit = t==0; subplot(2,2,[1 2]); plot(t, impulso_unit);  
>> escalon_unit = t>=0; subplot(2,2,3); plot(t, escalon_unit);  
>> rampa = t.*escalon_unit; subplot(2,2,4); plot(t, rampa);
```

Consulte para qué se emplean en MATLAB las funciones **dirac** y **heaviside**.

Ejercicio. Grafique las siguientes señales empleando las funciones **heaviside** y **linspace**:



1. $y_1(t) = 3u(t) - 2u(t - 2)$ para $-2 \leq t \leq 5$
2. $y_2(t) = 3r(t + 3) - 6r(t) + 3r(t - 3)$ para $-5 \leq t \leq 5$

6. Matemática simbólica en MATLAB

MATLAB también proporciona una serie de funciones para la resolución de problemas que involucren matemática simbólica. Como punto de partida se deben definir las variables que serán símbolos mediante el comando **syms**.

Experimento:

1. Defina las siguientes expresiones en MATLAB:

- $(x - y)(x - y)^2$
- $(x - 2)^3$
- $8x^2 - 30xy + 27y^2$
- $(x^3 - y^3)/(x - y)$

```
>> syms x y;  
>> p1 = (x-y) * (x-y) ^2  
>> p2 = (x-2) ^3  
>> p3 = (8*x^2-30*x*y+27*y^2)  
>> p4 = (x^3-y^3) / (x-y)
```

2. Expanda expresiones usando la función **expand()**

```
>> expand(p1)  
>> expand(p2)
```

3. Factorice expresiones usando la función **factor()**

```
>> factor(p3)
```

4. Simplifique expresiones usando la función **simplify()**

```
>> simplify(p4)
```

5. Sustituir valores en expresiones usando la función **subs()**: p5(x = 5) y p6(x,y=2)

```
>> p5 = (x-3) * (4*x^2-12*x+9)  
>> subs(p5,x,5)  
>> p6 = (4*x^2-12*x*y+9*y^2)  
>> subs(p6,y,2)
```

6. Derive expresiones usando la función **diff()**

```
>> p7 = 3*x^-2  
>> diff(p7)  
>> diff(p7,2)  
>> p8 = x/(sin(x) + cos(x))
```



```
>> diff(p8)
>> simplify(p8)
>> p9 = 3*x^-2 + 4*y^-4
>> diff(p9, x)
```

7. Integre expresiones usando la función **int()**

```
>> p10 = x^2
>> int(p10, x)
>> int(p10, 1, 2)
```

8. Evalúe límites usando la función **limit()**

```
>> p11 = sin(x)/x
>> limit(p11, x, 0)
>> limit(p11, x, 0, 'left')
```

9. Solucione ecuaciones usando la función **solve()**: $x^2 - 2x - 4 = 0$

```
>> p12 = x^2 - 2*x - 4 == 0;
>> solve(p12)
```

10. Con lo visto anteriormente, ¿Cómo haría usted para resolver el sistema: $\begin{cases} -y + 3x = 2 \\ y - 2x = 5 \end{cases}$?

7. Programación en MATLAB

MATLAB permite programar mediante un lenguaje de programación completo con características avanzadas. Con MATLAB es posible:

- a) Escribir código en scripts (.m) y funciones para automatizar tareas y reutilizar código.
- b) Usar condicionales (if-else), bucles (for y while) y switch para control de flujo.
- c) Soporta clases y objetos, permitiendo programar en un paradigma orientado a objetos.
- d) Permite crear interfaces gráficas (GUI) con App Designer y GUIDE.

La sintaxis básica de programación se muestra en la Tabla 15.

Estructura	Sintaxis	Descripción
Salida de texto	<code>disp(X)</code>	Permite desplegar un arreglo en pantalla. Por ejemplo: <code>nombre = 'Profesor';</code> <code>disp('Saludos');</code> <code>disp(nombre);</code>



Entrada de texto	<pre>user_entry = input('prompt') user_entry = input('prompt', 's')</pre>	Permite ingresar datos por teclado. Ejemplo: <pre>Nom=input('Nombre: ','s') disp('Su nombre es: ') disp(Nom)</pre>
Condicional	<pre>if expression1 statements1 elseif expression2 statements2 else statements3 end</pre>	Implementa condicionales. Ejemplo <pre>if nota<3 disp('Insuficiente') elseif nota<4 disp('Bueno') else disp('Excelente') end</pre>
Caso	<pre>switch switch_expr case case_expr statement, ..., statement case {case_expr1, case_expr2, case_expr3, ...} statement, ..., statement otherwise statement, ..., statement end</pre>	Permite el empleo de casos. Ejemplo: <pre>switch opcion case 1 disp('Dia de la semana'); case {2,3,4,5} disp('Dia de la semana'); case {6,7} disp('Fin de semana'); otherwise disp('Opcion invalida'); end</pre>
Ciclo para	<pre>for x=initval:endval statements end for x=initval:stepval:endval statements end</pre>	Implementación de la estructura for. <pre>for letra='a':'z' disp('letra') end for i=-3:0.4:3 disp(i) end</pre>
Ciclo mientras	<pre>while expression statements end</pre>	Implementación de la estructura mientras. <pre>suma=0,i=0 while i<10 suma=suma+i; i=i+1; end</pre>

Tabla 15. Sintaxis básica en MATLAB

Para codificar un programa en Matlab se tiene que invocar el editor, lo cual se hace con el comando **edit** o con el botón al cual está asociado que se muestra en la Figura 6.

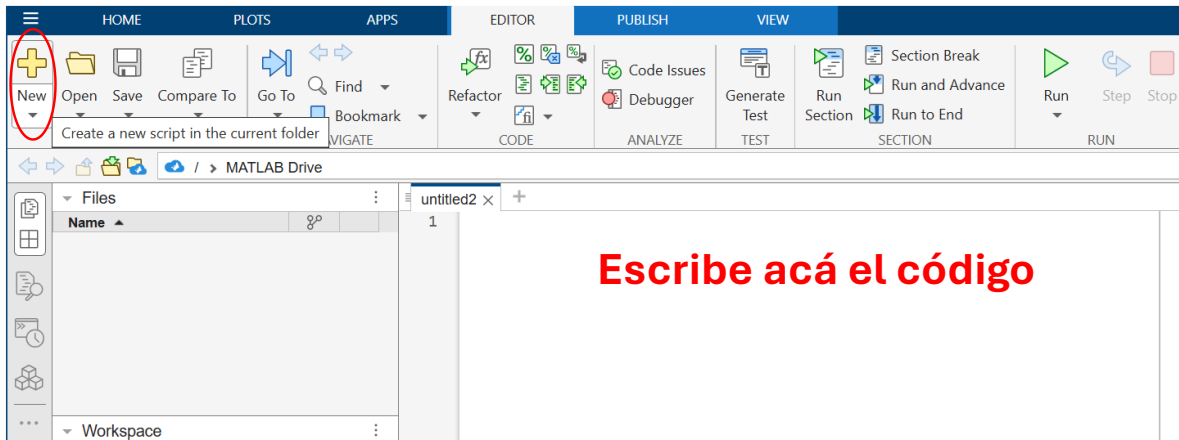


Figura 6. Editor de código en MATLAB

Como primer paso para programar un script en MATLAB, escriba el código de su script en el editor como se muestra en la Figura 7. A continuación, como segundo paso, guarde el archivo con un nombre que tenga la extensión .m (Ejemplo: test1.m). Finalmente, como tercer paso, ejecute el script presionando el botón RUN o la tecla F5. ¿Qué funcionalidad tiene el código? ¿Qué salida genera?

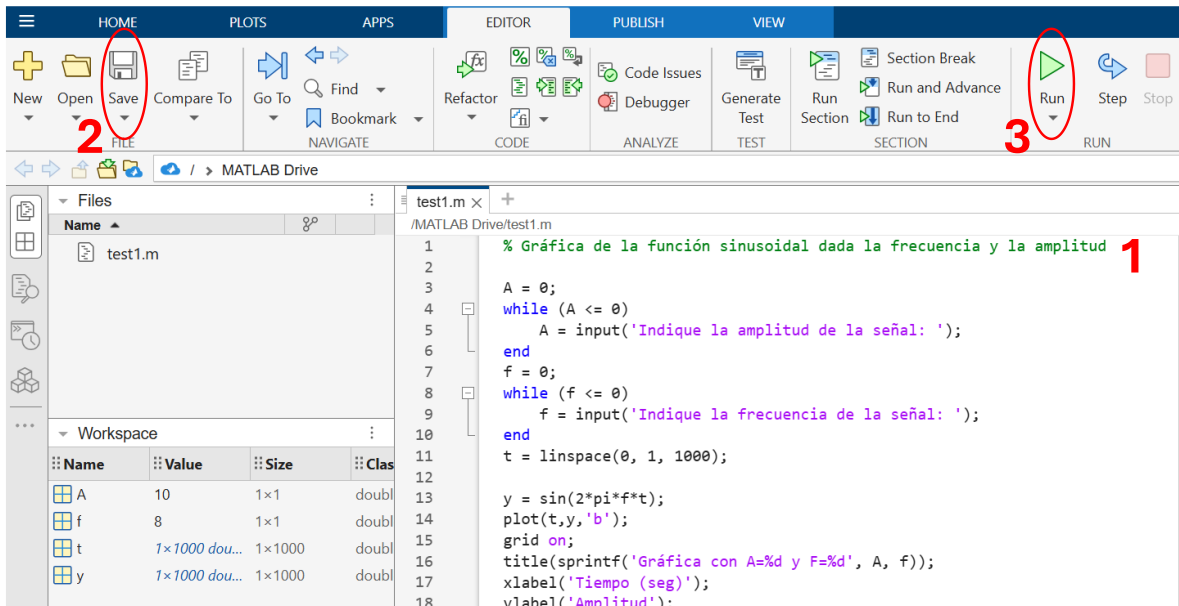


Figura 7. Creando y ejecutando un script en MATLAB